

NicaExplorer — Modelo de datos en Cloud Firestore

Modelo documental (NoSQL) de la base de datos y notas sobre normalización.

Proyecto: NicaExplorer (`com.lospuntoycoma.nicaexplorer`) · **Equipo:** Los Punto y Coma

Base de datos: Cloud Firestore — **NoSQL documental** (proyecto `nica-explore`)

Datos consultados: 8 de octubre de 2026, vía API REST `/api/v1/*` · **Versión del catálogo:** 2026-10-08T19:34:39Z

App: versionCode 5 / versionName 1.1.3 (release pública v1.1.3)

1. Objetivo y alcance

Este documento describe el modelo de datos de NicaExplorer, almacenado en **Cloud Firestore**, una base de datos **NoSQL orientada a documentos**. Se presentan:

- el **diagrama documental** (colecciones, documentos y campos);
- las **referencias** entre colecciones (equivalentes lógicos a claves foráneas);
- la **estructura de cada documento** con sus campos;
- una **nota sobre normalización (2FN)**, aclarando su alcance en un modelo NoSQL.

2. Nota metodológica: base de datos no relacional

Firestore **no es una base de datos relacional**: no tiene tablas, filas ni columnas, ni impone claves foráneas, *joins* ni esquema fijo. Su estructura es:

- Colección** → agrupa documentos de un mismo tipo (equivale conceptualmente a una «tabla»).
- Documento** → unidad de datos identificada por un **ID** (equivale a un registro, pero con campos anidables).
- Campo** → par clave-valor; puede ser escalar, **lista** o **mapa anidado**.
- Referencia** → un campo que guarda el ID de un documento de otra colección (no es una FK física).

Por este motivo, el diagrama se presenta como un **árbol de colecciones y documentos** (no como un diagrama entidad–relación con claves primarias y foráneas). Las **referencias** se señalan con una etiqueta azul.

3. Resumen del catálogo actual

Colección	ID del documento	N.º de documentos	Descripción
ciudades	cityId	4	Identidad, historia e imágenes de cada ciudad.
lugares	lugarId	42	Monumentos y sitios turísticos (con consejos de turismo responsable).
rutas	rutaId	4	Recorridos turísticos con paradas en lugares y comercios.
comercios	id	8	Restaurantes, cafeterías y negocios locales.
categorias_lugares	id	10	Catálogo jerárquico de categorías de lugares.
categorias_comercios	id	45	Catálogo jerárquico de categorías de comercios.
usuarios	uid	5	Perfiles y roles de los usuarios registrados.
solicitudes_comercios	id	1	Solicitudes de incorporación de comercios.
valoraciones	tipo_refId_uid	5	Estrellas otorgadas por los usuarios a lugares y comercios.
meta	catalogo	1	Versión del catálogo (<code>updatedAt</code>).

Total del catálogo: 4 ciudades, 42 lugares, 4 rutas, 8 comercios, 10 + 45 categorías, 5 usuarios, 5 valoraciones y 1 solicitud.

4. Diagrama documental (colecciones y documentos)

Cada nodo es una colección; el texto tras `/` es el ID del documento. Debajo se listan sus campos, con las listas marcadas con `[]` y las referencias resaltadas.

Cloud Firestore · proyecto nica-explore

ciudades `/cityId`

nombre · lema · descripcion · historia · departamento · imageUrl · galeria[] · latitud · longitud · monumentCount · activo

lugares `/lugarId`

`cityId` → ciudades · `categoria` → categorias_lugares · nombre · afluencia · descripcion · historia · anioConstruccion · modeloUnity · consejosResponsables[] · imageUrl · galeria[] · latitud · longitud · activo

rutas `/rutaId`

`cityId` → ciudades · nombre · descripcion · duracionEstimada · notaDuracion · objetivos[] · `paradas[]` → lugares | comercios · imageUrl · activo

comercios `/id`

`cityId` → ciudades · `categoria` → categorias_comercios · `propietarioUid` → usuarios · nombre · descripcion · ciudad · direccion · horario · telefono · whatsapp · correo · redesSociales[] · servicios[] · productos[] · infoAdicional · imageUrl · logoUrl · galeria[] · aprobado · activo

categorias_lugares `/id`

`categoriaPadre` → categorias_lugares · nombre · descripcion · icono · orden · activo

categorias_comercios `/id`

`categoriaPadre` → categorias_comercios · nombre · descripcion · icono · orden · activo

usuarios `/uid`

nombre · correo · rol · fechaRegistro

solicitudes_comercios `/id`

`cityId` → ciudades · `userId` → usuarios · nombreNegocio · categoria · direccion · descripcion · telefono · whatsapp · nombreResponsable · correoResponsable · estado · fechaSolicitud

valoraciones `/tipo_refId_uid`

tipo · `refId` → lugares | comercios · `uid` → usuarios · `cityId` → ciudades · estrellas

meta `/catalogo`

updatedAt (versión del catálogo que la app consulta)

[] = campo de tipo lista (arreglo embebido) · → = referencia a otra colección

5. Referencias entre colecciones

En Firestore no hay claves foráneas: cada referencia es un campo que guarda el ID de otro documento.

Colección origen	Campo	Colección destino	Tipo
lugares	cityId	ciudades	1 → N
lugares	categoria / categoriaPadre	categorias_lugares	N → 1
rutas	cityId	ciudades	1 → N
rutas	paradas[] (lugar:id / comercio:id)	lugares · comercios	N → N
comercios	cityId	ciudades	1 → N
comercios	categoria / categoriaPadre	categorias_comercios	N → 1
comercios	propietarioUid	usuarios	N → 1
categorias_lugares / categorias_comercios	categoriaPadre	la misma colección	auto-referencia
solicitudes_comercios	cityId / userId	ciudades · usuarios	N → 1

Colección origen	Campo	Colección destino	Tipo
valoraciones	refId (según tipo)	lugares · comercios · ciudades	N → 1 (polimórfica)
valoraciones	uid / cityId	usuarios · ciudades	N → 1

6. Estructura de los documentos (campos)

Tipos según la definición del panel (`NicaResources.php`) y los modelos de la app.

6.1. `ciudades/{cityId}`

Campo	Tipo	Notas
cityId	string	ID del documento (slug, p. ej. <code>juigalpa</code>).
nombre / lema / descripcion / historia / departamento	string / text	Identidad y textos.
imagenUrl / galeria	image / images	Portada y carrusel.
latitud / longitud / monumentCount / orden / activo	float / int / bool	Coordenadas y control.
nombreEn, lemaEn, descripcionEn, historiaEn	string / text	Campos bilingües (inglés).

6.2. `lugares/{lugarId}`

Campo	Tipo	Notas
lugarId	string	ID del documento (slug).
cityId	string (referencia)	→ <code>ciudades.cityId</code> .
categoria / categoriaPadre	string (referencia)	→ <code>categorias_lugares</code> (por nombre).
nombre / descripcion / historia / anioConstruccion	string / text	Textos del lugar.
afluencia	enum	<code>BAJA</code> / <code>MODERADA</code> / <code>ALTA</code> .
consejosResponsables	lista	Consejos de turismo responsable.
modeloUnity / imagenUrl / galeria	string / image / images	Visor 3D e imágenes.
latitud / longitud / orden / activo	float / int / bool	Coordenadas y control.
...En (nombre, categoria, descripcion, historia)	string / text	Campos bilingües.

6.3. `rutas/{rutaId}`

Campo	Tipo	Notas
rutaId / cityId	string	ID y referencia a <code>ciudades</code> .
nombre / descripcion / duracionEstimada / notaDuracion	string / text	Identidad y duración.
objetivos	lista	Objetivos de la ruta.
paradas	lista	Referencias <code>lugar:id</code> / <code>comercio:id</code> .
imagenUrl / orden / activo	image / int / bool	Presentación y control.
...En	string / text / lista	Campos bilingües.

6.4. `comercios/{id}`

Campo	Tipo	Notas
id / cityId	string	ID y referencia a <code>ciudades</code> .
categoria / categoriaPadre	string (referencia)	→ <code>categorias_comercios</code> .
propietarioUid	string (referencia)	→ <code>usuarios.uid</code> (cuenta COMERCIO).
nombre / descripcion / ciudad / direccion / horario / diasAtencion	string / text	Identidad y ubicación.
telefono / whatsapp / tieneWhatsapp / correo / redesSociales	string / bool / email / lista	Contacto.
servicios / productos / infoAdicional	lista / text	Oferta del negocio.

Campo	Tipo	Notas
imagenUrl / logoUrl / galeria	image / images	Imágenes.
latitud / longitud / orden / activo / aprobado	float / int / bool	Ubicación y estado.
...En	string / text / lista	Campos bilingües.

6.5. `categorias_lugares/{id}` y `categorias_comercios/{id}`

Campo	Tipo	Notas
id	string	ID del documento (slug).
nombre / nombreEn	string	Nombre (bilingüe).
categoriaPadre	string (referencia)	Auto-referencia a la categoría superior.
descripcion / icono / orden / activo	text / string / int / bool	Presentación.

6.6. `usuarios/{uid}`

Campo	Tipo	Notas
uid	string	ID de Firebase Authentication (ID del documento).
nombre / correo / rol / fechaRegistro	string / email / enum / timestamp	Perfil y rol (USUARIO / ADMIN / EDITOR / AUDITOR).

6.7. `solicitudes_comercios/{id}`

Campo	Tipo	Notas
id / cityId / userId	string	ID y referencias a <code>ciudades</code> y <code>usuarios</code> .
nombreNegocio / categoria / direccion / descripcion / horario	string / text	Datos del negocio.
telefono / whatsapp / redesSociales / nombreResponsable / correoResponsable	string / email	Contacto y responsable.
estado / fechaSolicitud	enum / timestamp	pendiente / aprobada / rechazada.

6.8. `valoraciones/{tipo_refId_uid}`

Campo	Tipo	Notas
tipo	enum	CIUDAD / LUGAR / COMERCIO.
refId	string (referencia)	ID del objeto valorado (lugares, comercios o ciudades).
uid / cityId	string (referencia)	→ <code>usuarios.uid</code> · <code>ciudades.cityId</code> .
estrellas	int	Valoración 1–5.

6.9. `meta/catalogo`

Campo	Tipo	Notas
updatedAt	timestamp	Versión que la app consulta para detectar cambios.

7. Nota sobre normalización (2FN) en un modelo NoSQL

Firestore **no exige** las formas normales del modelo relacional. Aun así, con fines académicos, si se proyecta el modelo documental a un modelo lógico (colección = tabla, documento = fila, campo = atributo), se observa que **cada documento se identifica por un único ID** (salvo `valoraciones`, cuya clave se compone de `tipo + refId + uid`). Al no existir claves compuestas con atributos parcialmente dependientes, **el modelo cumple la 2FN**; las únicas observaciones provienen de datos *desnormalizados* de forma deliberada (ver sección 8).

Colección	Identificador	Dependencias principales	¿2FN?
ciudades	cityId	cityId → nombre, lema, descripcion, historia, ...	Sí
lugares	lugarId	lugarId → cityId, categoria, nombre, afluencia, ...	Sí
rutas	rutald	rutald → cityId, nombre, paradas, objetivos, ...	Sí
comercios	id	id → cityId, categoria, propietarioUid, nombre, ...	Sí
categorias_lugares / categorias_comercios	id	id → nombre, categoriaPadre, icono, orden	Sí
usuarios	uid	uid → nombre, correo, rol, fechaRegistro	Sí
solicitudes_comercios	id	id → cityId, userId, nombreNegocio, estado, ...	Sí
valoraciones	tipo + refId + uid	(tipo, refId, uid) → estrellas · refId → cityId (parcial)	Sí*
meta	catalogo	catalogo → updatedAt	Sí

* **valoraciones:** `cityId` depende de `refId` (parte de la clave), lo que en un modelo relacional puro sería una dependencia parcial. En Firestore es una **desnormalización deliberada** para calcular el ranking por ciudad sin consultar otras colecciones; `estrellas` sí depende de la clave completa.

8. Desnormalizaciones controladas (propias de NoSQL)

- Referencias por nombre:** `lugares.categoria` y `comercios.categoria` guardan el nombre de la categoría en lugar de su ID.
- Texto de ciudad duplicado:** `lugares.ciudad` y `comercios.ciudad` repiten el nombre ya referenciado por `cityId`.
- Listas embebidas:** `galeria`, `consejosResponsables`, `servicios`, `productos`, `redesSociales` y `paradas` se guardan como arreglos dentro del documento, en lugar de colecciones hijas.
- Campos bilingües:** las variantes en inglés (`nombreEn`, `descripcionEn`, `historiaEn`, `lemaEn`, `categoriaEn`, `categoriaPadreEn`, `serviciosEn`, `productosEn`, `infoAdicionalEn`) conviven con el español. Actualmente el catálogo tiene cargados sus campos en inglés.
- Paradas polimórficas:** `rutas.paradas` mezcla referencias `lugar:` y `comercio:` en un mismo arreglo.

9. Correspondencia con la aplicación

- La app **lee el catálogo** a través de la API REST, que lee Firestore.
- La app **escribe directamente en Firestore** lo propio del usuario: `usuarios`, `comercios` (propios), `valoraciones` y `solicitudes_comercios`.
- El **panel web** administra todas las colecciones con credenciales de administrador.
- Las **imágenes** no se guardan en Firestore: se alojan en un repositorio público de GitHub y en los documentos se conserva la URL.